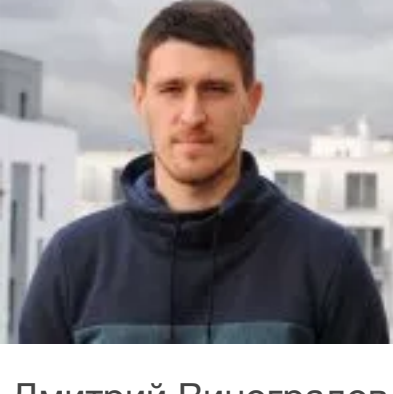


Language



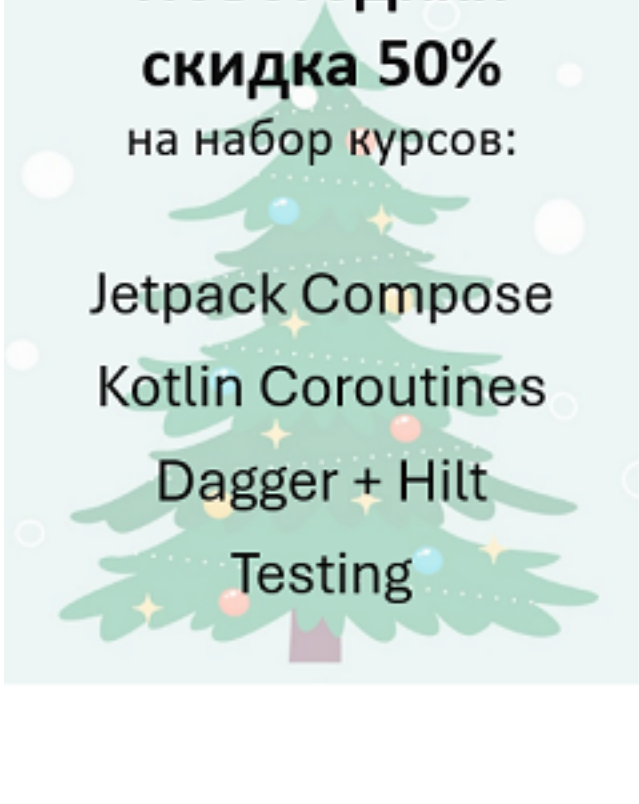
Автор сайта



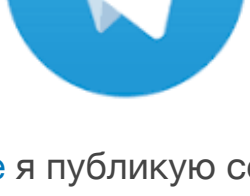
Дмитрий Виноградов

Подробнее можно [посмотреть](#) или [почитать](#).

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



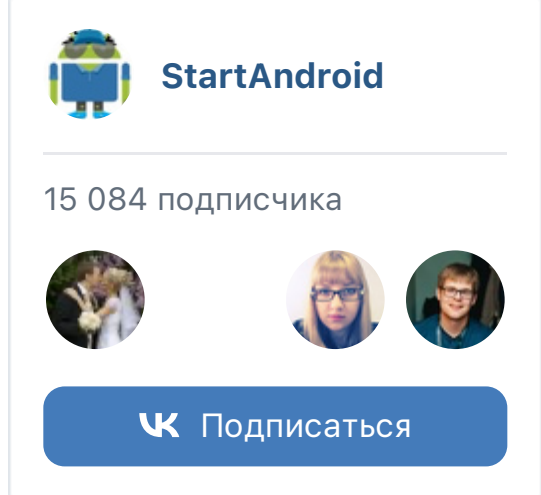
Социальные сети



В [канале](#) я публикую ссылки на интересные и полезные статьи по Android

В [чате](#) можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте



Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 8. Сабкомпоненты. Передача объектов

21 июня 2021

В этом уроке разбираемся, как передавать объекты в сабкомпоненты

В прошлом уроке я назвал вторым преимуществом сабкомпонентов более гибкую передачу объектов в компоненты.

Чтобы увидеть, как это работает, давайте пока забудем про сабкомпоненты и представим, что у нас есть только один компонент - AppComponent. Пусть этот компонент создает для нас MainActivityPresenter, когда мы открываем MainActivity.

Но что если нам для создания MainActivityPresenter нужен объект, который доступен только в момент открытия MainActivity. Это может быть, например, некий ID, который пришел в Intent-данных. И мы хотели бы передать этот ID в конструктор презентера, чтобы тот был готов отобразить нам данные, полученные с сервера или из БД по этому ID. Или вдруг нам по каким то причинам понадобится передать MainActivity в конструктор презентера.

В общем нам надо передать даггеру объект, чтобы он (даггер) использовал его в конструкторе презентера. Обычно мы такие объекты передаем при создании компонента. Но в нашем случае так сделать не получится. Объект, который мы хотим передать в презентер, становится доступен нам только в момент открытия MainActivity. А компонент AppComponent к этому моменту уже создан (в Application.onCreate). Мы уже не можем ничего ему передать.

Эта проблема решается использованием сабкомпонентов. При открытии MainActivity мы создаем сабкомпонент MainComponent, который создаст для нас презентер. Мы можем передать в этот сабкомпонент нужные нам объекты в момент его создания, и он передаст их в конструктор презентера.

Рассмотрим какие способы передачи объектов в сабкомпонент нам доступны.

Стандартный

Мы пока знаем только один способ создания сабкомпонента - метод в компоненте:

```
1 @Component(modules = [StorageModule::class, NetworkModule::class])
2 interface AppComponent {
3     fun getMainComponent(mainModule: MainModule): MainComponent
4
5
6 }
```

В прошлом уроке мы оставляли этот метод без аргументов. Но у нас есть возможность указать там модули, которые мы хотели бы вручить передать в сабкомпонент.

Теперь создание сабкомпонента выглядит так:

```
1 val mainComponent = appComponent.getMainComponent(MainModule(...))
```

В MainModule мы можем передать те объекты, которые мы хотели бы использовать при создании презентера. Все аналогично передаче объектов в обычный компонент.

У этого способа создания сабкомпонента есть недостаток. Мы можем передавать модули с нужными объектами, но не можем передавать объекты напрямую с помощью @BindInstance методов, как мы это делали в уроке про билдер.

Но есть и хорошая новость. У сабкомпонента, как и у обычного компонента, есть возможность создать кастомный Builder или Factory и в них включить @BindInstance методы.

Билдер

Кастомный билдер для сабкомпонента создается таким же способом, как и билдер для обычного компонента:

```
1 @Subcomponent(modules = [MainModule::class])
2 interface MainComponent {
3
4     @Subcomponent.Builder
5     interface Builder {
6         @BindInstance fun activity(activity: Activity): Builder
7         fun build(): MainComponent
8     }
9
10    fun getMainActivityPresenter(): MainActivityPresenter
11 }
```

Создаем интерфейс с аннотацией @Subcomponent.Builder, и в нем описываем, какие объекты хотим передать в сабкомпонент. Я в этом примере добавил в билдер возможность передать Activity, т.к. оно потребуется сабкомпоненту при создании MainActivityPresenter в MainModule:

```
1 @Module
2 class MainModule {
3
4     @Provides
5     fun provideMainActivityPresenter(..., activity: Activity): MainActivityPresenter {
6         return MainActivityPresenter(..., activity)
7     }
8
9 }
```

У сабкомпонента будет Activity и он сможет его использовать для создания презентера. А у MainModule теперь конструктор без аргументов, и нам не надо создавать его самим.

Мы настроили кастомный билдер у сабкомпонента. Как нам теперь получить этот билдер, чтобы создать сабкомпонент? Мы можем попросить его у компонента родителя:

```
1 @Component(modules = [StorageModule::class, NetworkModule::class])
2 interface AppComponent {
3
4     fun getMainComponentBuilder(): MainComponent.Builder
5
6 }
```

Т.е. мы у компонента просим не сам сабкомпонент, а его кастомный билдер, который мы описывали.

Мы получаем от компонента билдер и сами создаем сабкомпонент MainComponent:

```
1 val mainComponent = appComponent.getMainComponentBuilder()
2     .activity(this)
3     .build()
```

При этом мы передаем туда все, что нужно сабкомпоненту для создания объектов

Factory

Еще один способ передать объекты сабкомпоненту - это Factory. Ее мы уже проходили в уроке про билдеры. Для сабкомпонента все аналогично.

Описываем фабрику:

```
1 @Subcomponent(modules = [MainModule::class])
2 interface MainComponent {
3
4     @Subcomponent.Factory
5     interface Factory {
6         fun create(@BindInstance activity: Activity): MainComponent
7     }
8
9     fun getMainActivityPresenter(): MainActivityPresenter
10 }
```

В аргументах метода описываем объекты, которые хотим передать в сабкомпонент

В родительском компоненте описываем метод, который возвращает эту фабрику:

```
1 @Component(modules = [StorageModule::class, NetworkModule::class])
2 interface AppComponent {
3
4     fun getMainComponentFactory(): MainComponent.Factory
5
6 }
```

Получаем от компонента фабрику и создаем сабкомпонент:

```
1 val mainComponent = appComponent.getMainComponentFactory().create(this)
```

Inject

В примерах выше мы в компоненте описывали get методы для получения билдера (или фабрики) сабкомпонента. Но, если помните, в самом первом уроке мы говорили о том, что есть и другая возможность получить что-либо от компонента - inject метод. Если подзабыли, то посмотрите снова, чтобы понимать дальнейший текст.

Рассмотрим пример, когда мы хотим заинджектить билдер сабкомпонента в Activity.

Для этого мы в компоненте описываем inject метод:

```
1 @Component(modules = [StorageModule::class, NetworkModule::class])
2 interface AppComponent {
3
4     fun injectMainActivity(activity: MainActivity)
5
6 }
```

Напомним, мы передаем этому методу экземпляр Activity, и метод инджектит необходимые объекты в этот экземпляр.

В Activity добавляем поле с типом MainComponent.Builder и помечаем его аннотацией Inject:

```
1 class MainActivity : AppCompatActivity() {
2
3     @Inject
4     lateinit var mainComponentBuilder: MainComponent.Builder
5
6     override fun onCreate(savedInstanceState: Bundle?) {
7         super.onCreate(savedInstanceState)
8         setContentView(R.layout.activity_main)
9
10        (application as App).appComponent.injectMainActivity(this)
11
12        val mainComponent = mainComponentBuilder.activity(this).build()
13
14    }
15 }
```

Вызываем injectMainActivity метод у компонента, он поместит билдер в переменную mainComponentBuilder, и мы сможем его этот билдер использовать для построения сабкомпонента.

Все вроде правильно, но в таком виде это работать не будет. Не хватает одной детали. Обратите внимание, что сейчас в AppComponent нет вообще никакого упоминания о MainComponent. А оно, видимо, нужно по каким то причинам.

Это решается с помощью модулей, который используются в AppComponent. Надо в каком-нибудь из этих модулей в параметре subcomponents прописать MainComponent.

Это выглядит так:

```
1 @Module(subcomponents = [MainComponent::class])
2 class AppModule {
3
4     // ...
5
6 }
```

Этот модуль должен быть в списке модулей компонента, который инджектит билдер сабкомпонента MainComponent:

```
1 @Component(modules = [AppModule::class, StorageModule::class, NetworkModule::class])
2 interface AppComponent {
3
4     fun injectMainActivity(activity: MainActivity)
5
6 }
```

get vs inject

Есть ли разница между get и inject вариантами получения билдера сабкомпонента?

Официальная документация говорит нам следующее:

Using @Module.subcomponents is better since it allows Dagger to detect if the subcomponent is ever requested. Installing a subcomponent via a method on the parent component is an explicit request for that component, even if that method is never called. Dagger can't detect that, and thus must generate the subcomponent even if you never use it.

Комбинация inject метода и @Module.subcomponents является более предпочтительной, чем просто get метод. Потому что в случае get метода, даггер в любом случае будет выполнять кодогенерацию класса сабкомпонента, даже если мы забыли про этот метод и нигде в коде его не вызываем. А вот inject метод сможет определить, нужен ли сабкомпонент кому-то в коде или нет. И если не нужен, то даггер не будет выполнять лишнюю кодогенерацию.

[Назад](#) [Вперёд](#)

Комментарии

Inject — Вадим 23.06.2021 17:31
В первой картинке про Inject хотелось бы видеть больше кода внутри interface AppComponent
[Ответить](#)

RE: Урок 8. Сабкомпоненты. Передача объектов — Вадим 23.06.2021 18:09
"Все вроде правильно, но в таком виде это работать не будет. Не хватает одной детали. Обратите внимание, что сейчас в AppComponent нет вообще никакого упоминания о MainComponent. А оно, видимо, нужно по каким то причинам." - Это верно при условии, если удалить
fun getMainComponentBuilder(): MainComponent.Builder
из примера с билдером. Это было бы очевиднее, если бы вместо // ... в начале секции про Инджект был бы код
[Ответить](#)

RE: Урок 8. Сабкомпоненты. Передача объектов — Dmitry Vinogradov 28.08.2021 23:23
Убрал все // ... , чтобы исключить неопределенность
[Ответить](#)

Double inject — dev_course 18.11.2021 19:01
Получается, что мы можем заинджектить в поля MainActivity одно из двух:
а) или билдер с помощью AppComponent
б) или MainActivityPresenter с помощью MainComponent .
Если попробовать сделать оба действия одновременно, то словим креш:
class MainActivity : AppCompatActivity() {
 @Inject
 lateinit var mainComponentBuilder: MainComponent.Builder
 @Inject
 lateinit var presenter: MainActivityPresenter
 override fun onCreate(savedInstanceState: Bundle?) {
 ...
 (application as App).appComponent
 .injectMainActivity(this)
 mainComponentBuilder
 .context(this)
 .build()
 .injectMainActivity(this)
 }
}
То при первом же инжекте с помощью AppComponent попытается создать билдер, но и так же попробует создать MainActivityPresenter, хотя AppComponent не умеет его создавать.
Но из-за того, что мы хотим presenter заинджектить с помощью MainComponent (на самом деле инжектом с помощью MainComponent), то помечаем поля MainActivity аннотацией @Inject.
В итоге конечно же будет креш, говорящий о том, что Даггер не смог создать инстанс презентера.
Мораль:
- Либо использовать первый метод с getBuilder() методами и стараться чистить код от ненужных модулей (потенциальная проблема - излишек потребляемой постоянной памяти)
- Либо писать сильно больше кода для того, чтобы сделать Inject билдера в активности. Явная проблема - мы будем хранить ссылку на билдер в поле класса активности, что приведет к чуть большему потреблению RAM.
ИМХО, getBuilder()/getFactory() способ мне показался намного лаконичнее, понятнее, удобнее и экономичнее (с точки зрения RAM)
[Ответить](#)

🔄 Обновить список комментариев
📰 RSS лента комментариев этой записи

Добавить комментарий

Тема

☹️ 😄 😊 😋 😌 😍 😎 😏 😐 😑 😒 😓 😔 😕 😖 😗 😘 😙 😚 😛 😜 😝 😞 😟 😠 😡 😢 😣 😤 😥 😦 😧 😨 😩 😪 😫 😬 😭 😮 😯 😰 😱 😲 😳 😴 😵 😶 😷 😸 😹 😺 😻 😼 😽 😾 😿 😽 😿 😺 😻 😼 😽 😾 😿 😽 😿 😺 😻 😼 😽 😾 😿

Отправить

Comments

